

Doosri Raay: First Commit submission form answers

WeMakeDevs x AWS Builder Center, First Commit (17-20 Sep 2026), Ship It track. Prepared 20 Sep 2026.
Text in <ANGLE BRACKETS> is a placeholder to fill before pasting. Repository:
<https://github.com/abhay-codes07/doosri-raay>

Team

Field	Answer
Team leader's WeMakeDevs username *	<ABHAY_WEMAKEDEVs_USERNAME> (from wemakedevs.org/home)
Second team member's WeMakeDevs username	<RAJAT_WEMAKEDEVs_USERNAME>
Third team member's WeMakeDevs username	(leave blank unless a third member joined)
Fourth team member's WeMakeDevs username	(leave blank)
Team leader's GitHub *	https://github.com/abhay-codes07
Second team member's GitHub	https://github.com/rajatnagda45
Third team member's GitHub	(blank)
Fourth team member's GitHub	(blank)
Team leader's LinkedIn *	< https://www.linkedin.com/in/ABHAY >
Second team member's LinkedIn	< https://www.linkedin.com/in/RAJAT >
Third team member's LinkedIn	(blank)
Fourth team member's LinkedIn	(blank)
Team leader's resume (public Google Drive link)	< https://drive.google.com/file/d/.../view?usp=sharing > Set sharing to 'Anyone with the link'. Needed for the Amazon Fast Track interviews.
Second team member's resume	< https://drive.google.com/file/d/.../view?usp=sharing >
Third team member's resume	(blank)
Fourth team member's resume	(blank)

Project

Field	Answer
Project title *	Doosri Raay ("second opinion"): the outsider agent against digital-arrest scams
Track you are submitting for *	Ship It (cloud deployed). The same project also uses the Build It open-source stack (Strands Agents SDK, AWS SAM, LocalStack, Cedar), so if the form allows both tracks, tick both; Ship It is the primary.
GitHub link to project *	https://github.com/abhay-codes07/doosri-raay Make the repository PUBLIC before submitting (Settings > General > Danger zone > Change visibility). It has a judge-facing README, 100+ commits from 18 to 20 September, docs/ with the architecture, prior art, AI-tools disclosure and demo shot list, and MIT licence.

Field	Answer
Deployed link to project	https://main.d2inambt66a14p.amplifyapp.com/try No sign-up needed: /try signs the judge into a seeded judge circle (parent tile on the left, guardian dashboard on the right) with a judge FAQ, live timers and a Reset button. Two more judge circles exist (judge2@ and judge3@demo.doosriraay.in) so judges never collide. Judge password: <JUDGE_PASSWORD> (only for the optional full sign-in experience at /signin).
YouTube video demo link *	<YOUTUBE_URL> (unlisted, 3:00). Structure: 0:00 about the project and the problem, 0:18 the parent tile, 0:33 Watch to Ladder in Step Functions, 1:03 classifier, 1:15 Puchho with Polly, 1:27 recovery case, 2:05 console tour of every AWS service, 2:29 sources verified, 2:49 what we learned. Shot list: docs/DEMO.md.

What does your project do? (required)

What problem does your project solve, and who is it for? *

Doosri Raay is a family-linked safety net for Indian parents against "digital arrest" and other high-loss scams, built for the adult son or daughter who lives away from home.

The problem. Indians lost Rs 22,495 crore to cyber fraud in 2025 (MHA). Senior citizens filed 1,03,488 complaints worth Rs 4,005 crore (MHA reply in the Rajya Sabha, 5 Aug 2026). "Digital arrest" alone has 2,97,727 complaints and Rs 4,057.7 crore lost since 2022, and in Karnataka only 5% of that money was recovered in 2025, falling to 2.2% in early 2026. The victim is typically a retired professional held on a video call for hours or days, told to keep it secret from the family, and walked through breaking fixed deposits and wiring the money by RTGS. We read 25 documented cases from 2025-26: in every case that was saved, the save came from an outsider (a bank manager, a daughter, a neighbour, the police) or from an accident like the phone dying. Victims who acted alone did so only after reading a newspaper, one of them 165 days in. Every scam app we found starts from the victim's phone: it listens, shows a red warning, and expects a person under authority pressure to act. The evidence says they will not.

What it does. Doosri Raay builds the outsider.

- Passive isolation ladder (the hero). The parent's app is a genuinely useful Panchang tile: tithi, weather, medicine reminders, a family photo. Opening it each morning is the check-in. If it is not opened by the deadline and a guardian's call goes unanswered, a Step Functions ladder escalates: second guardian, then a named neighbour with a script and the address, then 112 guidance. Nothing ever appears on the parent's screen and nothing is recorded. The scammer watching the phone sees nothing.
- Recovery case manager. After a loss, the guardian uploads the UPI or bank screenshots. Bedrock extracts the references, code validates them (12-digit UPI/IMPS or 16-22 character NEFT/RTGS), the guardian confirms each field beside the image, and the app produces the 1930 call script, an NCRP-compliant narrative (200+ characters, allowed character set), the state's e-Zero FIR threshold and the Money Restoration Module checklist, each with its cited source. Every human step is a Step Functions callback with a deadline that escalates to a guardian.
- Puchho ("ask"). Two buttons on the parent's tile: "someone says they are police/CBI/bank" plays the official I4C line in Hindi (Amazon Polly) and alerts the guardians; "someone says my son is in trouble" pings the son with a family code-word challenge.
- Covert SOS. A triple-tap on the date sends location to the guardians with no visible change on screen.
- Screenshot check (minor tool). One Bedrock call with a forced tool schema returns one of three states and never the word "safe".

Who it is for. Adult children who worry about a parent living alone, the parent who wants no lecture and no scary app, and the family circle (a second guardian, a son, a neighbour) that the scam is designed to cut off. Guardians are notification-only: they never get the parent's balances, credentials or account access.

What we did differently from existing tools (Truecaller Family, Google's fake-call verification, PhonePe Protect, hackathon projects like Kavach, Rakshak and SwarVed): zero action required from the victim, no warning on the parent's screen at all, a validated end-to-end recovery pipeline through NCRP, e-Zero FIR and the refund module, hashed and re-verifiable sources behind every legal fact, and a deployed product with an evaluation harness.

How did you use AWS in your project? (required)

Build it: AWS open source stack. Ship it: AWS services. *

Ship It: one SAM stack in ap-south-1 (infra/template.yaml), nine services doing product work and five for operations.

- Amazon Bedrock. Claude Sonnet 4.6 (global.anthropic.claude-sonnet-4-6) with Claude Haiku 4.5 as the fallback, both through global cross-region inference profiles from Mumbai, via the Converse API. The classifier makes a single Converse call with forced tool use (toolChoice pinned to a tool schema) so the output is always an enum of three states, an enum of tactics and capped strings; the screenshot or message is passed as untrusted content. The recovery agent uses Bedrock for vision extraction into a tool schema and for the NCRP narrative, which code re-checks against the confirmed database rows so a hallucinated reference can never reach a complaint.
- AWS Step Functions (Standard). Three state machines. Watch is started by every check-in with the next deadline in its input; when it expires without a newer check-in it starts Ladder, so the missed-check-in trigger is itself an execution in the console (no scheduler). Ladder and RecoveryCase model every human step as `lambda:invoke.waitForTaskToken` with `TimeoutSecondsPath` read from the execution input, so the same definition runs with 15-minute rungs in production and 45-second rungs in the demo.
- AWS Lambda (Python 3.12). Six functions from one CodeUri: `api`, `classify-worker` (invoked asynchronously so `POST /analyze` returns 202 and the client polls), `ladder-task` (writes the task item with its token and sends Web Push), `watch-check`, `ladder-status`, and `recovery-agent` as a container image for the Strands SDK. X-Ray tracing is on, with the X-Ray SDK patching `boto3` so `DynamoDB`, `S3`, `Bedrock` and `Step Functions` calls appear as segments.
- Amazon API Gateway (HTTP API). Every route behind a Cognito JWT authorizer; the caller's circle is derived from their own profile item, never from the request. Stage throttling (20 rps, burst 50) and a per-user daily quota are the cost guards.
- Amazon Cognito. One user pool and app client; the frontend sends the ID token, which the authorizer validates against the client id; the `/demo` and `/try` pages hold a second (parent) identity in memory only.
- Amazon DynamoDB. One single-table design (profiles, circle members, check-ins, tasks carrying the Step Functions task token and a TTL, reports, cases, quotas), GSI for id lookups, point-in-time recovery.
- Amazon S3. One private bucket: presigned POST uploads with content-length-range (3.5 MB) and a Content-Type condition, SSE-S3, Block Public Access, 7-day lifecycle on screenshots, the cached Polly clip under `audio/`, and the hashed official sources under `sources/`.
- Amazon Polly. Neural Hindi voice (Kajal, Aditi fallback) renders the I4C advisory line for the Puchho button; synthesised once and cached in S3.
- AWS Amplify Hosting. Serves the Vite PWA from frontend/ with `amplify.yml` and an SPA rewrite rule; `VITE_*` variables come from the stack outputs.
- Operations: Systems Manager Parameter Store (SecureString for the VAPID private key), Amazon ECR (the recovery-agent image pushed by `sam deploy`), AWS Budgets (USD 20 and 50 alarms), CloudWatch Logs (14-day groups per function and state machine, ERROR level with execution data off so tokens are never logged), AWS X-Ray.

Build It (AWS open-source stack, used in the same project):

- Strands Agents SDK runs the recovery agent's tool loop; the tools (`extract_transactions`, `validate_fields`, `lookup_ezero_threshold`, `mrm_eligibility`, `draft_narrative`) are plain Python, capped at 12 tool calls, with a deterministic fallback path that runs the same functions if the agent fails.
- AWS SAM CLI builds and deploys the whole stack (`sam validate --lint`, `sam build --use-container`, `sam deploy`) and runs the API locally (`sam local start-api`).
- LocalStack serves `DynamoDB` and `S3` on the laptop (`docker-compose.yml`, `make local-up / local-bootstrap / local-api`); the backend honours `AWS_ENDPOINT_URL` and stubs `Step Functions` locally.

- Cedar is the authorization layer: backend/common/authz/policies.cedar decides which role may list which task kinds, complete a task, read a case or reset the demo; forbid rules carry @id annotations that become bilingual 403 messages, and a 16-row policy table is tested against the Rust engine (cedarpy) and a fallback evaluator of the same file.

We also followed the organisers' other techniques: every legal fact (e-Zero FIR thresholds, MRM rules, NCRP form rules) lives in a JSON data file with operator, value, source and quoted sentence; scripts/verify_sources.py downloads and SHA-256 hashes every cited government source and checks each quote against the live page (12 of 13 sources fetched, 25 of 25 quotes found on 20 Sep 2026), and the app shows the same hashes beside the facts.

Blog links

Up to four Builder Center blog links, one per member

<BUILDER_CENTER_BLOG_URL_ABHAY> (draft ready in docs/BLOG.md: "We built the outsider: why our scam project has no scam warning on the parent's screen")

<BUILDER_CENTER_BLOG_URL_RAJAT> (optional second post, e.g. "Deploying a Strands + Step Functions stack with SAM in one evening")

Contributions

Team leader's contributions *

Abhay Singh, team leader: research and architecture, backend, infrastructure.

- Research: the 25-case study of 2025-26 digital-arrest incidents and the 50-study review of intervention evidence that produced the pivot from victim-side detection to the outsider design; the sourced problem numbers; the prior-art review.
 - Architecture and contracts: the API, data-model and state-machine contracts (docs/), the single-table DynamoDB design, the self-renewing Watch design that replaces a scheduler.
 - Backend (backend/): the API handlers, the Bedrock classifier with forced tool use and untrusted-input wrapping, the Strands recovery agent with plain-code tools and deterministic fallback, the NCRP narrative re-check, rules-as-data (rules_data.json, patterns.json), the sources verification script and GET /sources, Cedar policies and authz layer, Polly clip cache, Web Push, quotas, the 230-test suite, the 70-item Hindi/English eval set and harness.
 - Infrastructure (infra/): the SAM template (Cognito, HTTP API with JWT authorizer, six Lambdas incl. the container image, hardened S3, DynamoDB, three Step Functions definitions with waitForTaskToken and TimeoutSecondsPath, log groups, Budgets, SSM), the Makefile, LocalStack local mode, the first-deploy checklist.
 - Docs: README, submission text, demo shot list, AI-tools disclosure.
- <ADJUST TO WHAT YOU ACTUALLY DID; keep it truthful to the commit history.>

Second team member's contributions

Rajat Nagda: frontend, deployment, demo.

- Frontend (frontend/): the Vite + React PWA with a custom bilingual Hindi-first auth flow, the parent Panchang tile with passive check-in, covert triple-tap SOS and Puchho, the guardian dashboard with task cards and the ladder status, case intake and case detail with sources and timers, Settings (family photo, holiday mode), the /demo split view and the no-sign-up /try judge route with the judge FAQ, the service worker and Web Push, amplify.yml.
 - Deployment and operations: AWS account and Bedrock model access, sam build and deploy, SSM VAPID key, Amplify Hosting with the SPA rewrite, seeding the demo and judge circles, uploading the hashed sources to S3, running the eval against Bedrock.
 - Demo: the 3-minute video, the Hindi copy review, the blog post.
- <ADJUST TO WHAT RAJAT ACTUALLY DID.>

Third team member's contributions

(blank)

Fourth team member's contributions

(blank)

Help us evaluate you: your feedback on the AWS services you used (required)

What you did not like and what could be better *

1. Bedrock from Mumbai means global inference profiles. Claude Sonnet 4.6 and Haiku 4.5 are reachable from ap-south-1 only through global.anthropic.* profile ids, and the IAM policy needs the profile and the underlying foundation-model ARNs in every region the profile can route to. Our stack is in ap-south-1; inference is not. The model page should say this up front. Amazon Nova 2 Sonic, which we wanted for the Hindi voice line, is not available in ap-south-1 at all and needs a bidirectional stream rather than Lambda, so Puchho uses Polly instead.
2. Bedrock is not in the Free tier and the Free plan has near-zero Bedrock quota. A student account needs the Paid plan enabled before the first Converse call succeeds, and the error until then does not look like a billing error. A one-line banner in the Bedrock console would save every hackathon team an hour.
3. Strands does not fit a zip Lambda comfortably. With boto3 and Pillow the bundle is large enough that we moved the recovery agent to a container image, which means ECR, Docker on every laptop, --platform linux/amd64, and BUILDX_NO_DEFAULT_ATTESTATIONS=1 because Lambda rejects the multi-manifest OCI index BuildKit produces by default. None of this is in the Strands Lambda guide. Strands also has no max-iterations option, so we added our own tool-call budget.
4. sam deploy --template infra/template.yaml is a trap. Passing the source template makes SAM skip the built template, so a PackageType: Image function has no ImageUri and the deploy fails with an unhelpful message. SAM could warn when an image function is deployed from an unbuilt template.
5. The HTTP API JWT authorizer wants the Cognito ID token. The access token has no aud claim, so every route returns 401 until you switch tokens, and neither Amplify's fetchAuthSession docs nor the authorizer docs say which one to send.
6. sam local start-api does not emulate the JWT authorizer, so local requests arrive with no claims; we had to add a dev-only identity fallback. LocalStack community edition has no Step Functions or Bedrock, so "the whole stack locally" is honest only for the API, DynamoDB and S3. A community-tier Step Functions emulator would complete the SAM + LocalStack story.
7. Amazon SES stays sandboxed for a hackathon (production access needs a review), and Amazon SNS SMS to Indian numbers needs DLT registration. Both are reasonable, but the consoles could say so before you spend an evening; we dropped e-mail and SMS for in-app tasks and Web Push.
8. Amplify Hosting needs an SPA rewrite rule for deep links (/case/<id> 404s until the regex rule to /index.html is added), and the rule cannot be expressed in amplify.yml; the Vite preset should add it by default. amplify.yml also does not pin Node, so a Vite 7 build can fail on the image's default Node.
9. waitForTaskToken with TimeoutSecondsPath is exactly the pattern for "a human has N minutes before we escalate" with a per-environment N, and we found it in the ASL reference rather than in a tutorial or a Workflow Studio template. With Standard workflows the timeout also does not close the task item on our side; we had to supersede stale tasks ourselves.
10. X-Ray with Tracing: Active alone shows one node. The DynamoDB, Bedrock and Step Functions segments need aws-xray-sdk patching in code, and the empty trace map does not say so.
11. Polly has no per-resource console object and no SecureString can be created by CloudFormation for the SSM parameter, so two of our services can only be shown in a video through an IAM statement and a placeholder overwrite.

12. Cedar has excellent semantics (forbid beats permit, policy ids in the decision) but the Python binding names policies `policy0..n`, so `@id` annotations have to be mapped by file order; first-class annotation lookup in the bindings would help.

What did you like about the AWS services you used? (required)

What worked well *

1. Step Functions is the product, not the plumbing. Watching the Watch execution expire, StartLadder fire and Ladder sit in Rung1GuardianCall waiting for a human is the whole idea in one screen. Standard workflows with `waitForTaskToken` and `TimeoutSecondsPath` let one definition run 15-minute rungs in production and 45-second rungs in the demo with no code change, and the console's execution graph is the best demo visual we have.
2. Bedrock Converse with forced tool use is structured output done right. One call with `toolChoice` pinned to a tool schema replaced a JSON parser, a retry loop and most of the prompt; the model can only answer inside our enums. Global cross-region profiles meant Sonnet 4.6 and Haiku 4.5 worked from ap-south-1 on the first try once access was enabled, and swapping the fallback model is one environment variable.
3. AWS SAM made a 28-resource stack (Cognito, HTTP API with a JWT authorizer, six Lambdas including a container image, DynamoDB, S3, three state machines, log groups, Budgets, SSM) a single template with policy templates like `DynamoDBCrudPolicy`, `sam validate --lint` caught our mistakes before any deploy, and `DefinitionSubstitutions` let the ASL files stay readable JSON with real ARNs injected at deploy.
4. The HTTP API JWT authorizer plus Cognito gave us per-route authentication with zero code: the caller's sub arrives in the request context and our handlers derive everything else from it, so no request can name a circle it does not belong to.
5. DynamoDB single-table design with TTL fit the domain perfectly: tasks carry their Step Functions token and expiry, reports and SOS items clean themselves up, and `PAY_PER_REQUEST` means the demo stack costs nothing while idle.
6. S3 presigned POST with a policy (content-length-range, Content-Type starts-with) was the cleanest way to let a browser upload straight to a private bucket with a hard size cap and no proxying through Lambda.
7. Amazon Polly's neural Hindi voice (Kajal) reads the I4C advisory line naturally; synthesising once and caching the MP3 in S3 made the feature effectively free.
8. Amplify Hosting connected to the GitHub repo with `amplify.yml` gave us preview builds on every push and a URL to put in the form; environment variables from the stack outputs made the frontend configuration a one-command step (`make env`).
9. The AWS open-source stack the Build It track pointed us to was genuinely useful: Strands let the tools be plain Python functions we could unit-test without the agent, and the model provider is swappable; LocalStack plus `sam local` gave a laptop loop for the API, DynamoDB and S3; Cedar's forbid-beats-permit semantics let us express "guardians are notification-only" and "the parent never sees covert tasks" as five policies instead of scattered conditionals, with the denial id surfacing as a bilingual message.
10. Onboarding: the Builder Center student credits and the Free tier credits covered the whole weekend, and the Bedrock playground earning an extra credit was a nice nudge to test model access before writing code.

Checklist before pasting

Notes

Before you paste:

- Make the GitHub repo public.
- Fill every `<PLACEHOLDER>`: usernames, LinkedIn, resumes, YouTube link, judge password, blog links. The live URL is already filled.
- Check the two contribution answers against what each of you actually did; the split above follows the roles in `docs/SUBMISSION.md`.

- The form can be edited until the deadline (the countdown on the submit page is authoritative), so submit early and refine.
- Team members must be student-verified on AWS Builder Center or the entry is not scored.